More Better Operators

Committee: ISO/IEC JTC1 SC22 WG21 EWG Evolution
Document Number: P0611R0
Date: 2017-03-18
Authors: Lawrence Crowl
Reply To: Lawrence Crowl, Lawrence@Crowl.org
Audience: EWG Evolution

Abstract

C++ can and should extend and improve its set of operators. Attention to several approaches and principles can reduce negative impacts from doing so. This paper provides two examples of improving operator support.

Contents

Introduction

When introduced, C was notable for its rich set of operators. Many subsequent languages have adopted those operators. They have also extended those operators, so that today the C operator set appears rather small. C++ is one of those subsequent languages, but one that has been more conservative in adding operators. See the Table of Operators for the current C and C++ operators.

The following table gives a sampling of extended operators in several languages. Named operators are particularly problematic, as sometimes they require more special syntax than operators, and sometimes they are not listed with the punctuation operators. I have omitted all compound assignment operators. Entries may not be correct or complete.

language	operators
C++	`:: .* ->* new new[] delete delete[] throw typeof decltype`
C#	`:: ?. ?[] checked unchecked delegate await Is As ?? =>`
D	`is <> <>= !< !> !<= !>= !<> !<>= in !in >>> ~ new delete cast ^^ ..`
Go	`[:] := <- &^`
Groovy	`new {} .& .@ ?. *. *: ** >>> .. ..> in instanceof as <=> =~ ==~`
Java	`instanceof >>>`
Perl	`** \ =~ !~ lt gt le ge <=> eq ne cmp ~~ .. ... =>`
PHP	`clone new ** @ instanceof === !== ?? . .=`
Python	`** // <> in 'not in' is 'is not'`
Ruby	`:: []= ** <=> === .eql? equal? =~ !~ .. ... defined?`
Swift	`... ..< ?? &+ &- &*` and extensible operators

Problem

How do we extend the operator set in a way that meets the need for concise syntax while not producing unparsable or unreadable expressions?

In addition, how do we remove operators that are problems?

Solution

There are several approaches to adding operators.

Extend the token set with new combinations of punctuation characters.

> Existing examples include `::` and `.*`. There exist proposals suggesting `<=>` and `->`.
>
> One problem with this approach is that what was formerly two tokens can become one token. For example, `a**b` changes meaning if one adds a Fortran-style `**` exponentiation operator.
>
> A careful examination of possible adjacent operators will reduce the likelyhood of problematic operator introduction. In general, one should avoid concatenation ambiguity between postfix operators and infix operators as well as between infix operators and prefix operators.
>
> Some rare ambiguities may be acceptable. For example, the code `a//*here*/b` changed meaning with the introduction of `//` comments. This change was not a significant problem in practice. Automated tools could help.

Extend the token set with new keywords.

> Examples include `new` and `throw`.
>
> One problem with this approach is that identifers in use in existing programs can suddenly become ill-formed. I had one customer strenuously objecting to introduction of the `export` keyword invalidating the public interface of his library.
>
> The policy of vetting keywords by searching open source software has reduced the severity of this problem.

Extend the 'fixity' of exiting operators.

> The most widely known example of this kind of extension is that of using the infix subtraction operator `-` as prefix negation operator. An example more specific to C and C++ is the extension of the infix operator `*` (multiply) to the prefix operator `*` (dereference).
>
> While all such 'fixity' extensions will yield parsable expressions, such expression may not be easily written or read. To avoid confusion, it is probably best that any given operator have at most two of the three fixities.
>
> A more subtle problem is adding a fixity that would naturally bring two tokens together that would be interpreted as a different combined token. The best example of this problem is the introduction of the `>` as a postfix 'close template argument list' operator. Closing two lists simultaneously led to the right shift operator.

Extend the basic character set to enable more operators.

> Some programming languages use Unicode (ISO 10646) as their base character set and allow use of the rich set of symbols within various code pages. While this approach is certainly viable for C and C++, it could cause compatiblity problems.
>
> C and C++ predate Unicode and presently depend on 'common' characters. These characters are mostly the non-national-use characters of the ISO 646 collection of character sets. However, C and C++ do use some ASCII-specific characters. Digraphs and trigraphs enable avoiding these characters when using a non-US variant of ISO 646 or when using other character sets, e.g. EBCDIC.
>
> There are two ASCII characters that are not in use by C and C++, `@` (commercial at) and `` ` `` (accent grave). Of those, `@` is in use by Objective C and Objective C++. These languages are meta-languages built on top of C and C++. In fact, other programmers may often need to build meta-languages on top of C and C++. Furthermore, they may need to compose two different meta language. That composition is possible in C and C++ today precisely because they do not use `@` and `` ` ``. I strongly recommend the C and C++ do not use those characters.

No approach above is likely to be sufficient alone. Furthermore, all approaches have the potential to invalidate existing code. If designers investigate potential invalidations and provide migration strategies, we can sanely extend the operator set.

Examples Of Change

In this section, I outline some changes to operators that I think would improve C and C++.

A Better Dereference Operator

The prefix dereference operator is a known and persistent problem for programmers. It complicates declaration and expression syntax. It occasionally requires grouping parentheses. It occasionally requires reading expressions inside-out.

Solution is a postfix dereference operator. The only parenthesis required are those for function calls. Declarations (and their corresponding expressions) are read strictly left-to-right, with the exception of the final type.

prefix	**postfix**
`int *v`	`int v^`
`int *a[6]`	`int a[6]^`
`int *f(int)`	`int f(int)^`
`int (*f)(int)`	`int f^(int)`
`int *(*f[6])(int)`	`int f[6]^(int)^`
`int *(*(*f)[6])(int)`	`int f^[6]^(int)^`

Once a postifix dereference operator is in place, the prefix dereference operator becomes redundant. It can be removed at some later time. After the prefix dereference operator is gone, that operator space could be reused for another purpose. This timeline leads to the following migration strategy.

Standards	**Programmers**
add `a^` as dereference	replace `*a` expressions with `a^`
deprecate `*a`	
remove `*a`	wait
add `a**b` for exponentiation and/or `*a` for something else	use that meaning

A Better Exclusive Or Operator

The expressions `a-b` and `-b` are natural pairs in the sense that the second can be rewritten as the first with a constant; `-b` is `0-b`.

The same is true of one's complement and exclusive or; `~b` is `ALLONES^b`. However, they don't use the same operator. They could though, as there is no current use of `~` as an infix operator. Indeed, PL/I took this approach where the operator was the ¬ symbol.

Changing the exclusive or operator to `~` would free up the infix `^` operator for other purposes. The most pressing use is exponentiation. It would not suprise me if most uses of binary `^` in C/C++ comments is as an exponentiation operator, not an exclusive or operator. This leads to the following migration strategy.

Standards	**Programmers**
add `a~b` as xor redefine `xor` to `~` redefine `xor_eq` to `~=`	replace uses of token `^` with `~`
deprecate `a^b`	
remove `a^b`	wait
add `a^b` as exponentiation add `pwr` as `^`	use `a^b` for exponentiation

Standards could either retain or remove token `xor`. The migration strategy for removal follows.

Standards	**Programmers**
wait	replace uses of token `xor` with `compl`
deprecate `xor`	
remove `xor`	do nothing

A New Exponentiation Operator

Both of the operator improvements above provide space for two different exponentiation (power) operators.

That space is needed because:

- The C `pow` function means different things in different libraries.
- Use of the `pow` function is not a clean mapping from typical mathematical expressions.
- A floating-point `pow` function is not a replacement for a integer power operation.

I prefer the `^` operator to the `**` operator, but `**` has the advange of not using national-use characters and hence needing no lexical work-arounds.

Miscellaneous Possibilitites

To round out the examples, let me provide some short hints at possibilities.

expression	**meaning**
`/a`	inversion `1/a`
`^a`	exp function `e^a`
`!!a`	logical identity `! !a`
`a!!b`	logical xor `!a~!b`
`a~~b`	logical xor `!a~!b`
`|a`	absolute value `a<0?-a:a`

Table of Operators

This table merges the operators of both C and C++. It extends the precedence levels as necessary to assign a shared consistent level.

operators	**prefix**	**infix**	**postfix**
`::`	1L scope (C++)	1L scope (C++)	
`()`	3R C-style cast (around type) 0 group (around expression)		2L functional cast (after type) (C++) 2L call (after expression)
`{} <%%>`	0 list (a literal in C)		2L functional cast (after type)
`[] <::>`	0 lambda (C++)		2L index
`. ->`		2L member	
`sizeof` `_Alignof` (C) `alignof` (C++)	3R type attribute		
`new new[]` `delete delete[]`	3R allocation (C++)		
`++ --`	3R pre-(in|dec)crement		2L post-(in|dec)crement
`~ compl`	3R bitwise not		
`! not`	3R logical not		
`.* ->*`		4L member pointer (C++)	
`*`	3R dereference	5L multiply	
`/`		5L divide	
`%`	*potential digraph conflict*	5L modulo	*potential digraph conflict*
`+`	3R promote	6L add	
`-`	3R negate	6L subtract	
`<< >>`		7L shift	
`<`		8L order ? template argument	
`>`		8L order	? template argument
`<= >=`		8L order	
`== != not_eq`		9L equality	
`& bitand`	3R address-of	10L bitwise and	
`^ xor`		11L bitwise xor	

`| bitor`		12L bitwise or	
`&& and`		13L logical and	
`|| or`		14L logical or	
`?:`		15R conditional (C) 16R conditional (C++)	
`throw`	16R throw (C++)		

`=`
`*= /= %=`
`+= -=`
`<<= >>=`
`&= ^= |=`
`and_eq xor_eq or_eq`
		16R assignment	

`,`		argument separator (within call) element separator (within list) 17L sequence (otherwise)	